ロール編集ダイアログUI設計書

メタデータ

title: ロール編集ダイアログUI設計書
category: UI設計
version: 1.0.0
last_updated: 2026-02-02
author: Development Team
tags: [system-admin, role-management, dialog, edit-form]

関連ドキュメント

- ロール管理画面
- ロール追加ダイアログ
- 機能管理ダイアログ
- ロール管理API仕様
- UIポリシー

---

1. 概要

1.1. 目的

システム管理者が既存ロールの情報を編集するためのモーダルダイアログです。ロール名、機能、初期画面機能名、状態を変更できます。

1.2. 表示条件

- ロール管理画面のテーブル行内「編集」ボタンをクリック

- System Admin権限が必要

1.3. 画面イメージ



---

2. ダイアログ構成

2.1. レイアウト

```
┌──────────────────────────────────────┐
│ ロール編集                      [×]  │
├──────────────────────────────────────┤
│                                      │
│  ロールCD *                          │
│  ┌──────────────────────────────┐    │
```



2.2. モーダル仕様

コンポーネント: `<Modal>`

仕様:

項目	値
タイトル	「ロール編集」
幅	600px
位置	画面中央
マスク	背景を暗くする
キーボードESC	閉じる（確認なし）
マスククリック	閉じない

3. 入力項目

3.1. ロールCD（読み取り専用）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「ロールCD」
必須表示	○（`*` マーク表示）
読み取り専用	○
背景色	`token.colorBgContainerDisabled`
初期値	選択したロールのロールCD

注意事項:

- ロールCDは主キーのため変更不可
- フォームには含まれるが、編集不可
- バリデーション対象外

実装例:

```
<Form.Item
  name="role"
  label="ロールCD"
  tooltip="ロールCDは変更できません"
>
  <Input
    readOnly={true}
    style={{
      backgroundColor: token.colorBgContainerDisabled,
      cursor: 'not-allowed'
    }}
  />
</Form.Item>
```

3.2. ロール名（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「ロール名」
必須	○
プレースホルダー	「ロール名を入力してください」
最大文字数	100文字
初期値	選択したロールのロール名

バリデーション:

```
{
  required: true,
  message: 'ロール名を入力してください'
},
{
  max: 100,
  message: 'ロール名は100文字以内で入力してください'
}
```

3.3. 機能名（読み取り専用、クリック可能）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「機能名」
必須	×
初期値	選択したロールの機能名（権限アイコン付き）
読み取り専用	○
クリック可能	○

背景色	`token.colorBgContainer`
カーソル	`pointer`

動作:

1. フィールドをクリック
2. 機能管理ダイアログを開く（directモード）
3. DBから最新のロール情報を取得して表示
4. 機能を変更して「機能を更新」をクリック
5. 即座にDBに反映される（ロール編集ダイアログの「保存」ボタンとは独立）
6. ダイアログを閉じる
7. 機能名フィールドが更新される
8. 初期画面機能名の選択肢が更新される

注意事項:

- 機能管理ダイアログでの変更は即座にDBに反映される
- ロール編集ダイアログの「保存」ボタンとは独立した処理
- 「キャンセル」をクリックしても機能の変更は維持される

実装例:

```
<Form.Item
  name="functionName"
  label="機能名"
>
  <Input
    readOnly={true}
    placeholder="—"
    onClick={() => handleOpenFunctionModalDirect(currentMaster)}
    style={{
      cursor: 'pointer',
      backgroundColor: token.colorBgContainer
    }}
  />
</Form.Item>
```

3.4. 初期画面機能名（条件付きSelect）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「初期画面機能名」
必須	×
プレースホルダー	「選択してください」
初期値	選択したロールの初期画面機能名
無効化条件	機能名が未設定または「－」の場合
クリア可能	○

選択肢:

- 機能名フィールドで設定されている機能のリスト
- 権限アイコン（Ⓦ と Ⓡ）を除去した機能名を表示
- 複数機能の場合、それぞれが選択肢になる

例:

- 機能名が `Ⓦシステム管理者メニュー / Ⓡ実験管理者メニュー` の場合
- 選択肢:
  - 「システム管理者メニュー」
  - 「実験管理者メニュー」

動的な活性/非活性制御:

```
<Form.Item
  noStyle
  shouldUpdate={(prevValues, currentValues) =>
    prevValues.functionName !== currentValues.functionName
  }
>
  {({ getFieldValue }) => {
    const functionNameValue = getFieldValue('functionName');
    const isFunctionDisabled = !functionNameValue || functionNameValue === '－';

    return (
```

```
      <Form.Item
        name="initialScreenFunctionName"
        label="初期画面機能名"
      >
        <Select
          placeholder="選択してください"
          disabled={isFunctionDisabled}
          options={functionOptions}
          allowClear
        />
      </Form.Item>
    );
  }}
</Form.Item>
```

3.5. 状態（必須）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「状態」
必須	○
初期値	選択したロールの状態

選択肢:

```
[
  { value: true, label: 'アクティブ' },
  { value: false, label: '非アクティブ' }
]
```

---

4. アクションボタン

4.1. 配置

位置: モーダルフッター（右寄せ）

レイアウト:

```
<Space>
  <Button onClick={handleModalCancel}>キャンセル</Button>
  <Button type="primary" onClick={handleModalOk}>保存</Button>
</Space>
```

4.2. キャンセルボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「キャンセル」
タイプ	`default`

動作:

1. モーダルを閉じる
2. フォームをリセット
3. 確認なし

注意事項:

- 機能管理ダイアログで行った変更は維持される（DBに既に反映済み）
- キャンセルしても機能の変更は元に戻らない

4.3. 保存ボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「保存」

タイプ	`primary`
ローディング	送信中は `true`

動作フロー:

1. フォームバリデーション実行
2. バリデーションエラーがある場合、エラーメッセージ表示
3. ローディング状態に変更
4. ロール情報を更新（`updateRole` API）
    - ロール名
    - 初期画面機能名
    - 状態
5. 成功時:
    - モーダルを閉じる
    - データ再読み込み
    - 成功メッセージ表示: `"ロールを更新しました"`
6. 失敗時:
    - エラーメッセージ表示
    - モーダルは開いたまま

実装例:

```
const handleModalOk = async () => {
  try {
    const values = await form.validateFields();

    await updateRole(currentMaster.role, {
      roleName: values.roleName,
      initialScreenFunctionName: values.initialScreenFunctionName || null,
      isActive: values.isActive,
    });

    await fetchRoleMasters();
    message.success('ロールを更新しました');
```

```
      setIsModalVisible(false);
      form.resetFields();
    } catch (error) {
      console.error('エラー:', error);
      const errorMessage = error instanceof Error ? error.message : 'エラーが発生しました';
      message.error(errorMessage);
    }
  };
```

---

5. API仕様

5.1. ロール情報更新

エンドポイント: `PUT /api/v1/systemManagement/role/:roleId`

リクエストヘッダー:

```
Content-Type: application/json
Authorization: Bearer {session_token}
```

リクエストボディ:

```
{
  "roleName": "プロバイダー（更新）",
  "initialScreenFunctionName": "プロバイダーメニュー",
  "isActive": true
}
```

レスポンス（成功時）:

```
{
  "success": true,
  "data": {
    "roleId": "provider",
    "roleName": "プロバイダー（更新）",
    "initialScreenFunctionName": "プロバイダーメニュー",
    "isActive": true,
```

```
    "updatedAt": "2026-02-02T09:00:00.000Z"
  },
  "meta": {
    "timestamp": "2026-02-02T09:00:00.000Z",
    "requestId": "req_abc123"
  }
}
```

5.2. ロール詳細取得（機能管理ダイアログ用）

エンドポイント: `GET /api/v1/systemManagement/role/:roleId`

リクエストヘッダー:

```
Authorization: Bearer {session_token}
```

レスポンス（成功時）:

```
{
  "success": true,
  "data": {
    "roleId": "provider",
    "roleName": "プロバイダー",
    "functionName": "Ⓦプロバイダーメニュー",
    "initialScreenFunctionName": "プロバイダーメニュー",
    "functionWritePermissions": "プロバイダーメニュー",
    "functionIds": ["010000"],
    "writableFunctionIds": ["010000"],
    "isActive": true,
    "updatedAt": "2026-02-02T09:00:00.000Z"
  },
  "meta": {
    "timestamp": "2026-02-02T09:00:00.000Z",
    "requestId": "req_abc123"
  }
}
```

---

6. バリデーション

6.1. クライアントサイドバリデーション

必須チェック:

- ロール名
- 状態

形式チェック:

- 最大文字数: ロール名（100文字）

6.2. サーバーサイドバリデーション

データ整合性チェック:

- 初期画面機能名が選択された機能のリストに含まれているか
- ロールCDが存在するか

---

7. エラーハンドリング

7.1. エラーメッセージ一覧

エラーコード	メッセージ	表示場所
`VALIDATION_ERROR`	「ロール名を入力してください」	フォームフィールド
`RESOURCE_NOT_FOUND`	「指定されたロールが見つかりません」	トースト
`SYSTEM_ERROR`	「エラーが発生しました」	トースト

7.2. エラー表示方法

フォームフィールドエラー:

```
form.setFields([
  {
    name: 'roleName',
    errors: ['ロール名を入力してください'],
```

```
    },
  ]);
```

トーストメッセージ:

```
message.error('エラーが発生しました');
```

---

8. ユーザーフロー

8.1. 正常フロー（機能変更あり）

```
[ロール管理画面]
    ↓ 行内「編集」ボタンクリック
[ロール編集ダイアログ] 表示
    ロールCD: "provider"（読み取り専用）
    ロール名: "プロバイダー"
    機能名: "Ⓦプロバイダーメニュー"
    初期画面機能名: "プロバイダーメニュー"
    状態: アクティブ
    ↓ 機能名フィールドクリック
[機能管理ダイアログ] 表示（directモード）
    DBから最新のロール情報を取得
    ↓ 機能を変更（例: システム管理者メニューを追加）
    ↓ 「機能を更新」クリック
API呼び出し: updateRolePermissions()
    ↓ 成功（DBに即座に反映）
[ロール編集ダイアログ] に戻る
    機能名: "Ⓦプロバイダーメニュー / Ⓦシステム管理者メニュー"
    初期画面機能名: 選択肢が更新される
    ↓ 初期画面機能名を変更（オプション）
    ↓ 「保存」ボタンクリック
API呼び出し: updateRole()
    ↓ 成功
[ロール管理画面]
    データ再読み込み
    成功メッセージ表示: "ロールを更新しました"
```

8.2. 正常フロー（機能変更なし）

```
[ロール管理画面]
    ↓ 行内「編集」ボタンクリック
[ロール編集ダイアログ] 表示
    ↓ ロール名を変更: "プロバイダー" → "プロバイダー（更新）"
    ↓ 「保存」ボタンクリック
API呼び出し: updateRole()
    ↓ 成功
[ロール管理画面]
    データ再読み込み
    成功メッセージ表示: "ロールを更新しました"
```

8.3. キャンセルフロー（機能変更後）

```
[ロール編集ダイアログ]
    ↓ 機能名フィールドクリック
[機能管理ダイアログ] 表示
    ↓ 機能を変更
    ↓ 「機能を更新」クリック
API呼び出し: updateRolePermissions()
    ↓ 成功（DBに即座に反映）
[ロール編集ダイアログ] に戻る
    機能名: 更新された
    ↓ ロール名を変更
    ↓ 「キャンセル」ボタンクリック
[ロール管理画面]
    機能の変更はDBに反映済み（維持される）
    ロール名の変更は破棄される
```

---

9. 追加ダイアログとの違い

項目	追加ダイアログ	編集ダイアログ
タイトル	「ロール追加」	「ロール編集」
ロールCD	入力可能	読み取り専用
初期値	空	選択したロールの情報
機能管理モード	temporary（メモリに一時保存）	direct（DB直接更新）
機能変更のタイミング	「保存」ボタンクリック時	「機能を更新」ボタンクリック時

キャンセル時の機能	メモリをクリア	DB反映済み（維持）
APIエンドポイント	`POST /api/v1/systemManagement/role`	`PUT /api/v1/systemManagement/role/:roleId`

---

10. アクセシビリティ

10.1. キーボード操作

Tab順序:

1. ロールCD（読み取り専用、スキップ可能）
2. ロール名入力
3. 機能名入力（クリック可能）
4. 初期画面機能名選択
5. 状態選択
6. キャンセルボタン
7. 保存ボタン

ショートカットキー:

- Enter: 保存ボタンのクリック（フォーカスが入力フィールドにない場合）
- Escape: モーダルを閉じる

10.2. ARIA属性

```
<Modal
  title="ロール編集"
  open={isModalVisible}
  onCancel={handleModalCancel}
  aria-labelledby="role-edit-dialog-title"
  aria-describedby="role-edit-dialog-description"
>
  <Form form={form} layout="vertical">
    <Form.Item
```

```
        label="ロールCD"
        name="role"
        tooltip="ロールCDは変更できません"
      >
        <Input
          readOnly={true}
          aria-label="ロールCD（読み取り専用）"
          aria-readonly="true"
        />
      </Form.Item>
    </Form>
  </Modal>
```

---

11. 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-02-02	初版作成	Development Team